# Multimodal Medical Assistant for Image-Text Clinical Triage

*HPPCS[04] Capstone Report*

**Abstract**

This project implements a clinician-facing multimodal assistant that combines a radiology scan or histopathology slide with a structured clinical note to produce triage guidance, image–text correlation, evidence-linked summaries, and interactive follow-up chat. The clinical aim is to reduce missed cross-modal cues by forcing vision and text models to share one case record. Compact local models are used deliberately: MedGemma 4B for vision and Llama 3.2 (3B, with 1B fallback) for text reasoning, chosen for memory efficiency on a typical 16 GB CPU laptop or Google Colab (CPU or free T4 GPU). Ollama auto-adapts to CPU or GPU; the app unloads the vision model after analysis so both LLMs share limited RAM/VRAM. Modalities are restricted to radiology and pathology with auto domain detection. LangGraph sequences analysis and follow-up re-triage. The CLI takes --text and --image paths, prints triage summaries, continues into follow-up chat, and writes conversation.json in the Codebase directory; optional --tui offers the same pipeline with interactive path prompts. Label-free evaluation metrics are printed after every run.

## 1. Introduction

- Context: Imaging and notes are usually read separately; this system binds them in one LangGraph state so triage uses visual findings and note context together.
- Motivation: Local MedGemma + Llama on Ollama keep PHI on-premises and match HPPCS[04].

## 2. Problem Statement

Given one radiology or pathology image and one clinical note, the system must detect domain, correlate image findings with the note, assign triage (emergency / urgent / soon / routine), answer follow-up questions with updated recommendations, and report evaluation metrics. Out-of-scope photographs (dermatology, fundus, clinical snapshots) are rejected.

## 3. Objectives

- Provide a CLI (and optional TUI) so clinicians pass a note and image path and receive triage summaries.
- Return image-referenced findings and contextual explanations from the clinical note.
- Support adaptive follow-up with Llama re-triage of impression, differential, recommendations, and next questions.
- Auto-detect radiology versus pathology and reject out-of-scope images.

- De-identify text and images before any model call.
- Emit evaluation metrics (correlation, robustness, explanation quality, satisfaction) on every run.
- Write the conversation output as conversation.json in the Codebase directory.

**4. Methodology**

- Stack: Python 3.12, LangGraph, Ollama, MedGemma 4B, Llama 3.2 3B, runtime_profile.py.
- Workflow: de-identify → MedGemma (domain + findings) → Llama entities → Llama triage JSON → summary → follow-up re-triage; metrics via evaluation.py.

*Pipeline: Note+Image → Privacy → MedGemma → Llama → conversation.json → follow-up chat.*

**5. System Design / Implementation**

Architecture (layers): Presentation (main.py CLI and optional tui_app.py) → Application (medical_assistant.py) → Orchestration (graph_pipeline.py / LangGraph) → Models (ollama_client.py with runtime_profile.py for CPU/GPU) → Supporting modules (ingestion.py, privacy.py, evaluation.py, paths.py). After each run, medical_assistant.py writes conversation.json in the Codebase directory.

- main.py — CLI with --text and --image; follow-up chat after analysis; optional --tui; --device auto|cpu|gpu.
- runtime_profile.py — detects NVIDIA GPU vs CPU; tunes keep-alive, timeouts, and unload policy.
- graph_pipeline.py — four-node LangGraph; follow-up re-triage grounded in image and note.
- evaluation.py — correlation, robustness, explanation quality, and satisfaction metrics.
- tui_app.py — shared terminal summary and follow-up chat loop used by CLI and --tui.
- execution.txt — setup and run instructions for the submission.

**6. Observations and Results**

Evaluation is performed during a live CLI or TUI run: pass note and image paths, wait for analysis, continue follow-up chat at the You> prompt, then read printed metrics and Codebase/conversation.json. Faculty may supply their own note and image at demo time.

On representative radiology and pathology cases, MedGemma distinguishes domains; Llama returns structured triage (emergency / urgent / soon / routine) with impression, differential, and recommendations; image–note correlation links findings to the note; and follow-up questions trigger re-triage. Quantitative metrics from evaluation.py:

- Cross-modal correlation — image–note narrative presence and token overlap.
- Robustness — valid domain detection, de-identified processing, and field completeness.

- Explanation quality — checklist of clinical explanation fields (mapped to 1–5 Likert).
- User satisfaction — interface completeness proxy from triage, findings, chat, and prompts.

Demo command: python main.py --text <note.txt> --image <scan.jpg> from Codebase/.

## 7. Conclusion and Future Work

- Summary of contributions: dual-LLM multimodal triage with auto domain detection, privacy gate, CLI (and optional TUI), adaptive follow-up, and evaluation metric outputs.
- Possible extensions: larger GPU serving when hardware allows, and reader-study Likert collection from clinicians.

## 8. References


- HPPCS[04] capstone brief.
- Google Health AI Developer Foundations, MedGemma model card.
- Meta Llama 3.2 model card; Ollama library (medgemma, llama3.2).
- LangGraph documentation; Wikimedia Commons teaching images in sample_data/.


## Acknowledgement


Thanks to HAAI++ faculty for the HPPCS[04] brief and to the MedGemma, Llama, LangGraph, and Ollama communities. Teaching images are public; accompanying notes are fictional.